announce-reactive: local stack test guide

How to start the whole stack on the development machine, where to reach each part, which member and staff accounts to use, and a walk-through that exercises every service once. Rendered 2026-10-25 from `scripts/stack-test-guide.py`; the accounts and ports below are the ones `scripts/start-stack.sh` prints when it is done.

1. Start and stop

From the repository root (a worktree is fine, the script builds from the checkout it lives in):

```
scripts/start-stack.sh                # builds what is missing on the first run, then starts everything
scripts/start-stack.sh --build        # rebuild every jar first (after pulling code)
scripts/start-stack.sh --without-order # skip the order module (auctions, orders, payments, shipping)
scripts/start-stack.sh --search=noop   # do not index into OCS; search stays as it is
scripts/stop-stack.sh                 # stop what start-stack.sh started (pid files)
scripts/stop-stack.sh --all           # also stop announce services started by hand
scripts/stop-stack.sh --all --infra   # and stop the docker/cqrs containers
```

What the script does, in order:

1. Refuses to start if a service of a previous run is alive or one of the ports is taken, and names the process.
2. Checks MySQL `announce_reactive_db` on localhost:3306 (user `announce`). The database is built by `db-reactive/sql/build_announce_reactive_db.sh` and is never the `litemall` database.
3. Checks the six `docker/cqrs` containers (EventSourcingDB, Kafka, Elasticsearch, OCS indexer, searcher, suggest) and runs `docker compose up -d` when one is missing, then waits for the health checks.
4. Builds the backends in one Maven reactor (`-am`, so db-reactive, cqrs-reactive and the api modules come from this checkout, not from a stale SNAPSHOT in `~/.m2`) and the two edges with their SPAs (`webapp` profile, npm). Build logs: `~/.announce-stack/logs/build-*.log`.
5. Starts announce-management, order, messaging, the customer edge and the admin edge with Eureka, the config server and discovery switched off; the edges get the backend URLs as `--announce.routes.*` overrides. messaging has no bootable jar and runs under `mvn spring-boot:run`.
6. Waits until each service answers over HTTP and prints the URLs and accounts.

Java 21 is taken from `/usr/lib/jvm/java-21-openjdk-amd64` unless `JAVA_HOME` is set. Logs and pid files live in `~/.announce-stack` (override with `STACK_HOME`).

2. Addresses

What	URL	Notes
Customer SPA and edge	`http://localhost:8090/`	The marketplace as members see it. Port 8080 is usually taken on this machine, hence 8090 (`PORT_CUSTOMER`).
Admin SPA and edge	`http://localhost:8091/`	Moderation, catalogue, members, trade lists, support queue (`PORT_ADMIN`).
announce-management	`http://localhost:8085/`	Announces, categories, brands, crawled listings, search indexing. Direct calls answer 401 without `X-Service-Token`: go through an edge.
order	`http://localhost:8083/`	Auctions, orders, payments (fake provider), shipping (fake carrier). Same 401 rule.
messaging	`http://localhost:8086/`	Conversations and WebSockets. Same 401 rule.
EventSourcingDB UI	`http://localhost:3000/`	Token `secret`. The event store behind every aggregate.
Elasticsearch	`http://localhost:9200/`	Index `announce_index`.
OCS searcher / indexer / suggest	`localhost:8534 / 8535 / 8536`	The search tier the SPA queries through the customer edge.

What	URL	Notes
Kafka	`localhost:9094`	Topics `announce.v1`, `auction.v1`, `order.v1`, `messaging.v1`, `messaging.live`.
Health	`/actuator/health` on 8085, 8083, 8086	Reports DOWN locally because the Redis and Kafka-binder indicators cannot reach their hosts. The services work; the script treats 503 as up.

3. Accounts

3.1 Members (customer SPA, `http://localhost:8090/login`*)*

Username	Password	Id	What it is good for
`user123`	`user123`	1	The seed member. Owns the 22 published sample announces (ids 2000001 and up), 3 drafts and 1 withdrawn one. Use it as the seller side.
`sdcheck1792845801`	`Descriptor-2026`	61	Created on 2026-10-24. Owns the published announce `f16dcaf4-0e66-4bff-be59-a98b95a07442` and one draft. Seller in the chat test below.
`exp21792849879`	`Descriptor-2026`	64	Created on 2026-10-25. Buyer who opened conversation `6523e1a7-3d2a-31c0-80c3-2c759a57b86c` with member 61.
new	your choice, 8+ chars		Register at `/register`: username (3 to 63 chars of `A-Z a-z 0-9 . _ @ -`), password, optional nickname and mobile. The customer edge creates the row in `announce_user`.

A member can only chat with another member about an announce, so every chat test needs two accounts in two browser sessions (a private window works). Member tokens last 2 hours and refresh through an httpOnly cookie.

3.2 Staff (admin SPA, `http://localhost:8091/login`*)*

Username	Password	Role	Permissions today
`admin123`	`admin123`	1 Super administrator	`*`: everything, including admins, roles and permissions.
`mall123`	`mall123`	2 Marketplace administrator	Logs in with `ROLE_ADMIN`. Meant for announces, catalogue, listings, members, regions.
`promotion123`	`promotion123`	3 Support administrator	Logs in with `ROLE_ADMIN`. Meant for the support queue and bulletins.

Permission codes are not enforced per route yet (worktree `staff-permissions`, steps 2 to 7 are still a guide, see `doc/staff-permissions-report.pdf`): any enabled staff account can use every screen except the super-only ones (admins, roles, permissions). The seed roles 2 and 3 still carry litemall permission strings. Treat `admin123` as the account for every staff test.

4. Walk-through: one pass over every service

Each block names the services it touches. Do them in order the first time; later you can jump to one.

4.1 Register and log in (customer edge)

1. Open `http://localhost:8090/`, click "Register", create a member (call it the buyer). You are logged in afterwards; the header shows the username as a dropdown.
2. Log out, log in as `user123` (the seller). "My announces" in the dropdown lists its 22 published announces and 3 drafts.

4.2 Post, publish, search (announce-management, OCS)

1. As the seller click "Post" and pick a kind. Goods: title, category of the goods tree, price, condition, sale type (FIXED_PRICE, AUCTION or GIVEAWAY). Job: employer, contract type, deadline, how to apply; no sale type. Event: start before end, in the future; venue and organizer.
2. Save: the announce is a DRAFT, visible only to you (`/announce/{id}` answers 404 to anyone else). Open it from "My announces" and click "Publish" with a validity in days.
3. Search for a word of the title at `/search`. With `--search=ocs` on, the search projection indexed it into OCS within a second or two; the facets show `kind` and `source`. Crawled listings from dba.dk and tradera.com appear in the same results with a source badge; their click goes out through `/out/{id}`.
4. Open the announce as a visitor (log out or a private window): status PUBLISHED, the kind-specific fields, no "Edit" button.

   Right now every crawled listing row is GONE (last crawl older than the 7-day grace), so no crawled hit shows up until `crawler/run.sh` runs and the file in `~/announce_crawled_data` is imported (announce-management does it on start and every poll).

4.3 Chat between two members (messaging, the shared announce descriptor)

1. As the buyer (second browser session) open a published announce of the seller, for example `http://localhost:8090/announce/f16dcaf4-0e66-4bff-be59-a98b95a07442`, and click "Message the seller". messaging asks announce-management for the announce over the shared `AnnounceApi` descriptor and opens one conversation per announce and buyer; you land on `/messages/{id}`.
2. Write a message and send it. In the seller's session the "Messages" entry of the dropdown lists the conversation; open it and reply. Both sides update live over the WebSocket.
3. As the buyer send an offer (the offer control in the composer, goods announces only). As the seller accept or decline it. An accepted offer publishes `OfferAcceptedV1` on `messaging.v1`; with the order module up it becomes an order.
4. Clicking "Message the seller" again on the same announce reopens the same conversation. On your own announce it answers "buyer is seller". On a draft or unknown id it answers "announce not found".
5. Support: in the inbox at `/messages` the "Contact support" form opens a SUPPORT conversation that waits until staff joins (4.5).

4.4 Trade (order module, fake payment and carrier)

1. Fixed price: as the buyer open a FIXED_PRICE announce and request to buy; the seller accepts from "Orders"; the order is AGREED. Auction: publish an AUCTION announce; the order module opens the auction from `announce.v1`; the buyer bids from the auction panel; a bid from the seller or below the increment is refused.

2. Payment and shipping use fake providers. The webhooks `POST /api/payments/webhook/fake` and `POST /api/shipping/webhook/fake` take the provider event as JSON with an HMAC-SHA256 hex signature of the raw body in `X-Fake-Signature` (secrets `payment.fake.secret` and `shipping.fake.secret` in the order configuration). The exact bodies and the full state machine are in `doc/order-context.md` and `doc/customer-trade-wiring-report.pdf`.
3. After the payment webhook the order is PAID and announce-management marks the announce SOLD (subscriber on `order.v1`). After carrier delivery and the grace period the payout is released and the order SETTLED. The "Orders" screen of both members and the admin "Orders" and "Payouts" lists follow along.

4.5 Staff (admin edge)

1. Log in at `http://localhost:8091/login` as `admin123`. The dashboard counts announces, crawled listings and members.
2. Announces: filter by kind, status, seller or text; withdraw or expire one with a reason. The member sees the new status at once; a withdrawn announce leaves the search index.
3. Catalogue: categories per kind and brands. Members: search, open one, change its status. Regions.
4. Crawled listings: trigger an import of a file path, a reindex, and read the inventory (per source, status, quality grade, dead links).
5. Support queue at `/support`: the WAITING list shows support conversations nobody picked up; "Join" moves one to OPEN and lets you answer. Switch the status dropdown to "Open" to see listing conversations between members; `/support/{id}` opens any thread read-only (close or block it). The two test conversations are on the last page of "Open" because older load-test rows come first.
6. Bulletins: post one to an audience; connected staff clients receive it over `/ws/staff`.
7. Auctions, Orders, Payouts: the trade lists over the order module; resolve a dispute from an order.

5. Looking under the hood

- Logs: `~/.announce-stack/logs/` with one file per service; messaging logs every outbound HTTP call at TRACE when you add `--logging.level.org.springframework.web.reactive.function.client.ExchangeFunctions=TRACE` to its arguments.
- Events: the EventSourcingDB UI at `http://localhost:3000/` (token `secret`) shows the subjects `/announce/{id}`, `/conversation/{id}`, `/auction/{id}`, `/order/{id}`, `/payment/{id}`, `/shipment/{id}`.
- Read models: MySQL `announce_reactive_db`, tables `announce`, `announce_goods_detail`, `crawled_listing`, `announce_conversation`, `announce_chat_message`, `announce_order` and the projection checkpoints in `cqrs_readside_progress`.
- Direct backend calls need the service token. Easiest: call through an edge with a member or staff bearer token (`POST /auth/login` then `Authorization: Bearer ...`).

6. Known quirks

- announce-management imports `dba_cars.json` (and any `dba_*.jsonl` / `tradera_*.jsonl` file) from `~/announce_crawled_data` on every start. It is idempotent; it just takes a few seconds and fills the log.
- Jars built before 2026-10-01 predate the service-token change and answer every proxied call with 401. Run `scripts/start-stack.sh --build` after pulling.
- Port 8080 is held by another application on this machine, so the customer edge runs on 8090 and the admin edge on 8091 (its own default is 8081). Override with `PORT_CUSTOMER` and `PORT_ADMIN`.
- The messaging end-to-end test suite stalls while a messaging instance of the stack uses the live database: stop the stack before `mvn -pl messaging test`.

- Headless browsers on this machine have no DNS, so external images in the SPA fail to load there; a desktop browser is fine.
- A dev EventSourcingDB that stops answering reads after broken replay streams: restart the container, wait for healthy, then start the services one by one (see `doc/messaging-tuning-so-far-report.pdf`).